

1. Write a menu-driven program in C to create a Double Linked List, display it in forward and backward directions, insert a node (at beginning, at specific position, at end), and delete a node (from beginning, from specific position, from end).

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    struct node *prev;
    int data;
    struct node *next;
};

struct node* create_list();
void display_forward(struct node *head);
void display_backward(struct node *head);
struct node* insert_node(struct node *head);
struct node* delete_node(struct node *head);
void free_list(struct node *head);

int main() {
    struct node *head = NULL;
    int choice;

    while (1) {
        printf("\n1. Create Double Linked List\n");
        printf("2. Display List (Forward & Backward)\n");
        printf("3. Insert a Node\n");
        printf("4. Delete a Node\n");
        printf("5. Exit\n");
        printf("Enter your choice (1-5): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                if (head != NULL) {
                    free_list(head);
                    head = NULL;
                }
                head = create_list();
                break;
            case 2:
                display_forward(head);
                display_backward(head);
                break;
            case 3:
                head = insert_node(head);
                break;
            case 4:
                head = delete_node(head);
                break;
            case 5:
```

```

        free_list(head);
        exit(0);
    default:
        printf("Invalid choice!\n");
    }
}
return 0;
}

struct node* create_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
        newNode->prev = NULL;
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            newNode->prev = ptr;
            ptr = ptr->next;
        }

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    return head;
}

void display_forward(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *ptr = head;
    printf("Forward Direction: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

```

```

void display_backward(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *ptr = head;
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    printf("Backward Direction: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->prev;
    }
    printf("\n");
}

struct node* insert_node(struct node *head) {
    int value, choice, pos;
    printf("Enter data of new node: ");
    scanf("%d", &value);

    struct node *newNode = (struct node*) malloc(sizeof(struct node));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (head == NULL) {
        return newNode;
    }

    printf("1. At Beginning\n2. At Specific Position\n3. At End\n");
    printf("Enter insertion type: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            newNode->next = head;
            head->prev = newNode;
            head = newNode;
            break;

        case 2: {
            printf("Enter Position (>= 1): ");
            scanf("%d", &pos);

            if (pos <= 1) {
                newNode->next = head;
                head->prev = newNode;
                head = newNode;
                break;
            }

            struct node *ptr = head;

```

```

        int count = 1;
        while (ptr->next != NULL && count < pos - 1) {
            ptr = ptr->next;
            count++;
        }

        newNode->next = ptr->next;
        newNode->prev = ptr;
        if (ptr->next != NULL) {
            ptr->next->prev = newNode;
        }
        ptr->next = newNode;
        break;
    }

    case 3: {
        struct node *ptr = head;
        while (ptr->next != NULL) {
            ptr = ptr->next;
        }
        ptr->next = newNode;
        newNode->prev = ptr;
        break;
    }

    default: free(newNode); return head;
}

printf("\nAfter Insertion:\n");
display_forward(head);
display_backward(head);
return head;
}

struct node* delete_node(struct node *head) {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return NULL;
    }

    int choice, pos;
    printf("1. From Beginning\n2. From Specific Position\n3. From End\n");
    printf("Enter deletion type: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: {
            struct node *temp = head;
            head = head->next;
            if (head != NULL) {
                head->prev = NULL;
            }
            free(temp);
            break;
        }
    }
}

```

```

}

case 2: {
    printf("Enter Position: ");
    scanf("%d", &pos);

    if (pos <= 1) {
        struct node *temp = head;
        head = head->next;
        if (head != NULL) {
            head->prev = NULL;
        }
        free(temp);
        break;
    }

    struct node *ptr = head;
    int count = 1;
    while (ptr != NULL && count < pos) {
        ptr = ptr->next;
        count++;
    }

    if (ptr == NULL) {
        printf("Position out of bounds.\n");
        return head;
    }

    if (ptr->prev != NULL) {
        ptr->prev->next = ptr->next;
    }
    if (ptr->next != NULL) {
        ptr->next->prev = ptr->prev;
    }
    free(ptr);
    break;
}

case 3: {
    struct node *ptr = head;
    if (ptr->next == NULL) {
        free(ptr);
        head = NULL;
        break;
    }
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    ptr->prev->next = NULL;
    free(ptr);
    break;
}

default: return head;

```

```
    }

    printf("\nAfter Deletion:\n");
    display_forward(head);
    return head;
}

void free_list(struct node *head) {
    struct node *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}
```

Output:

```
PS D:\DS\LAB4> gcc p1.c -o p1.exe
PS D:\DS\LAB4> .\p1.exe

1. Create Double Linked List
2. Display List (Forward & Backward)
3. Insert a Node
4. Delete a Node
5. Exit
Enter your choice (1-5): 1
Enter Data: 10
Do you have another data? (y/n): y
Enter Data: 20
Do you have another data? (y/n): y
Enter Data: 30
Do you have another data? (y/n): n

Enter your choice (1-5): 2
Forward Direction: 10 20 30
Backward Direction: 30 20 10

Enter your choice (1-5): 3
Enter data of new node: 15
1. At Beginning
2. At Specific Position
3. At End
Enter insertion type: 2
Enter Position (>= 1): 2

After Insertion:
Forward Direction: 10 15 20 30
Backward Direction: 30 20 15 10

Enter your choice (1-5): 4
1. From Beginning
2. From Specific Position
3. From End
Enter deletion type: 2
Enter Position: 3

After Deletion:
Forward Direction: 10 15 30

Enter your choice (1-5): 5
PS D:\DS\LAB4>
```

2. Write a program in C for concatenation of two double linked lists.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    struct node *prev;
    int data;
    struct node *next;
};

struct node* create_list();
void display(struct node *head);
struct node* concatenate(struct node *head1, struct node *head2);
void free_list(struct node *head);

int main() {
    struct node *head1 = NULL, *head2 = NULL, *head3 = NULL;

    printf("Enter elements for First Double Linked List:\n");
    head1 = create_list();

    printf("\nEnter elements for Second Double Linked List:\n");
    head2 = create_list();

    printf("\nFirst ");
    display(head1);

    printf("Second ");
    display(head2);

    head3 = concatenate(head1, head2);

    printf("\nConcatenated ");
    display(head3);

    free_list(head3);
    return 0;
}

struct node* create_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
```

```

        newNode->prev = NULL;
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            newNode->prev = ptr;
            ptr = ptr->next;
        }

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    return head;
}

void display(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *ptr = head;
    printf("List elements: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

struct node* concatenate(struct node *head1, struct node *head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;

    struct node *ptr = head1;
    while (ptr->next != NULL) {
        ptr = ptr->next;
    }
    ptr->next = head2;
    head2->prev = ptr;
    return head1;
}

void free_list(struct node *head) {
    struct node *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

```

Output:

```
PS D:\DS\LAB4> gcc p2.c -o p2.exe
PS D:\DS\LAB4> .\p2.exe
Enter elements for First Double Linked List:
Enter Data: 10
Do you have another data? (y/n): y
Enter Data: 20
Do you have another data? (y/n): n

Enter elements for Second Double Linked List:
Enter Data: 30
Do you have another data? (y/n): y
Enter Data: 40
Do you have another data? (y/n): n

First List elements: 10 20
Second List elements: 30 40

Concatenated List elements: 10 20 30 40

PS D:\DS\LAB4>
```

3. Write a program in C for merging two sorted linked lists into one sorted single linked list.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node* create_list();
void display(struct node *head);
struct node* merge_sorted(struct node *head1, struct node *head2);
void free_list(struct node *head);

int main() {
    struct node *head1 = NULL, *head2 = NULL, *merged = NULL;

    printf("Enter elements for First Sorted Linked List:\n");
    head1 = create_list();

    printf("\nEnter elements for Second Sorted Linked List:\n");
    head2 = create_list();

    printf("\nFirst ");
    display(head1);

    printf("Second ");
    display(head2);

    merged = merge_sorted(head1, head2);

    printf("\nMerged Sorted ");
    display(merged);

    free_list(merged);
    return 0;
}

struct node* create_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;
```

```

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            ptr = ptr->next;
        }

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    return head;
}

void display(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *ptr = head;
    printf("List elements: ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

struct node* merge_sorted(struct node *head1, struct node *head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;

    struct node *result = NULL, *tail = NULL;

    if (head1->data <= head2->data) {
        result = head1;
        head1 = head1->next;
    } else {
        result = head2;
        head2 = head2->next;
    }
    tail = result;

    while (head1 != NULL && head2 != NULL) {
        if (head1->data <= head2->data) {
            tail->next = head1;
            head1 = head1->next;
        } else {
            tail->next = head2;
            head2 = head2->next;
        }
        tail = tail->next;
    }

```

```

    }

    if (head1 != NULL) {
        tail->next = head1;
    } else {
        tail->next = head2;
    }

    return result;
}

void free_list(struct node *head) {
    struct node *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

```

Output:

```

PS D:\DS\LAB4> gcc p3.c -o p3.exe
PS D:\DS\LAB4> .\p3.exe
Enter elements for First Sorted Linked List:
Enter Data: 10
Do you have another data? (y/n): y
Enter Data: 30
Do you have another data? (y/n): y
Enter Data: 50
Do you have another data? (y/n): n

Enter elements for Second Sorted Linked List:
Enter Data: 20
Do you have another data? (y/n): y
Enter Data: 40
Do you have another data? (y/n): y
Enter Data: 60
Do you have another data? (y/n): n

First List elements: 10 30 50
Second List elements: 20 40 60

Merged Sorted List elements: 10 20 30 40 50 60

PS D:\DS\LAB4>

```

4. Write a program in C to create a circular linked list and count the number of even and odd numbers present in the linked list.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node* create_circular_list();
void display(struct node *head);
void count_even_odd(struct node *head);
void free_circular_list(struct node *head);

int main() {
    struct node *head = NULL;

    printf("Enter elements for Circular Linked List:\n");
    head = create_circular_list();

    printf("\n");
    display(head);

    count_even_odd(head);

    free_circular_list(head);
    return 0;
}

struct node* create_circular_list() {
    struct node *head = NULL, *ptr = NULL, *newNode = NULL;
    char ch;

    do {
        newNode = (struct node*) malloc(sizeof(struct node));
        if (newNode == NULL) {
            return head;
        }

        printf("Enter Data: ");
        scanf("%d", &newNode->data);
        newNode->next = NULL;

        if (head == NULL) {
            head = newNode;
            ptr = newNode;
        } else {
            ptr->next = newNode;
            ptr = ptr->next;
        }
    }
}
```

```

        printf("Do you have another data? (y/n): ");
        scanf(" %c", &ch);
    } while (ch == 'y' || ch == 'Y');

    if (ptr != NULL) {
        ptr->next = head;
    }

    return head;
}

void display(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct node *ptr = head;
    printf("Circular List elements: ");
    do {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    } while (ptr != head);
    printf("\n");
}

void count_even_odd(struct node *head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    int even = 0, odd = 0;
    struct node *ptr = head;

    do {
        if (ptr->data % 2 == 0) {
            even++;
        } else {
            odd++;
        }
        ptr = ptr->next;
    } while (ptr != head);

    printf("Count of Even numbers: %d\n", even);
    printf("Count of Odd numbers : %d\n", odd);
}

void free_circular_list(struct node *head) {
    if (head == NULL) return;
    struct node *ptr = head, *next;

    // Break the circle to free standard list
    ptr = head;

```

```

while (ptr->next != head) {
    ptr = ptr->next;
}
ptr->next = NULL;

ptr = head;
while (ptr != NULL) {
    next = ptr->next;
    free(ptr);
    ptr = next;
}
}

```

Output:

```

PS D:\DS\LAB4> gcc p4.c -o p4.exe
PS D:\DS\LAB4> .\p4.exe
Enter elements for Circular Linked List:
Enter Data: 12
Do you have another data? (y/n): y
Enter Data: 15
Do you have another data? (y/n): y
Enter Data: 20
Do you have another data? (y/n): y
Enter Data: 23
Do you have another data? (y/n): y
Enter Data: 30
Do you have another data? (y/n): n

Circular List elements: 12 15 20 23 30
Count of Even numbers: 3
Count of Odd numbers : 2

PS D:\DS\LAB4>

```